DL

Building Training Systems with PyTorch

Chapter 6 · Tensors, autograd, modules, data pipelines, training, evaluation, and persistence

Dr. A. Belcaid

Table of contents

0.1 The system we will build . . . 3

1 From arrays to tensors **4**
1.1 A tensor carries a contract . . . 4
1.1.1 Scalars, vectors, matrices, and higher-rank tensors . . . 5
1.2 Creating tensors . . . 5
1.2.1 From Python data . . . 5
1.2.2 Filled tensors . . . 6
1.2.3 Ranges and evenly spaced values . . . 6
1.2.4 Random tensors and reproducibility . . . 6
1.3 Moving between NumPy and PyTorch . . . 7

2 Tensor fluency **8**
2.1 Indexing and slicing . . . 8
2.1.1 Why preserving an axis matters . . . 8
2.2 Boolean masks . . . 9
2.3 Reshaping without changing the values . . . 10
2.3.1 `reshape`, `view`, and contiguity . . . 10
2.4 Adding, removing, and reordering axes . . . 11
2.5 Reductions and dimensions . . . 11
2.5.1 Values and indices . . . 12
2.6 Broadcasting . . . 12
2.6.1 Scalar with a vector . . . 13
2.6.2 Row vector with a matrix . . . 13
2.6.3 Column vector with a matrix . . . 13
2.6.4 Per-channel image normalization . . . 14
2.7 Practice: predict before running . . . 14

3 Autograd **15**
3.1 What the forward pass records . . . 15

3.2 Backward populates leaf gradients . . . 15
3.3 A batch loss becomes one scalar . . . 16
3.4 Gradients accumulate . . . 17
3.5 Disabling gradient recording . . . 17

4 Models as `nn.Module` 18
4.1 The module contract . . . 18
4.2 A Fashion-MNIST MLP . . . 19
4.2.1 Calling the model . . . 19
4.2.2 `Sequential` versus an explicit `forward` . . . 19
4.3 Registered parameters . . . 20
4.4 Device movement . . . 21

5 Data pipeline 21
5.1 `Dataset` and `DataLoader` solve different problems . . . 21
5.2 Loading Fashion-MNIST . . . 22
5.3 Inspect a sample first . . . 23
5.4 Batching and shuffling . . . 23
5.4.1 Loader performance options . . . 24
5.5 A minimal custom dataset . . . 25

6 Training and validation 25
6.1 Configure the objective and optimizer . . . 25
6.2 One optimization step . . . 26
6.3 Training for one epoch . . . 27
6.4 Validation . . . 27
6.5 Correct metric aggregation . . . 28
6.5.1 Confusion matrices and per-class behavior . . . 29
6.6 Complete epoch driver . . . 30
6.7 Practice: repair a broken loop . . . 31

7 Saving, restoring, and inference 31
7.1 Save the best validation state . . . 31
7.2 What a `state_dict` contains . . . 32
7.3 Restore on any chosen device . . . 32
7.3.1 Saving enough information to resume training . . . 33
7.4 Predict one image . . . 34

8 A complete minimal program 35

9 Debugging checklist 37
9.1 Data . . . 37
9.2 Model . . . 38
9.3 Gradients . . . 38
9.4 Evaluation . . . 39

10 Further reading 39

11 Chapter summary 39

PART IV · PYTORCH FUNDAMENTALS



Figure 1: A minimal Python-like ribbon passes through a translucent tensor cube, connecting Python programming to tensor computation. Original course illustration generated for this chapter.

Deep learning code is not only a formula translated into Python. A working system must keep track of tensor shapes, gradients, model state, batches, devices, evaluation behavior, and saved parameters. **PyTorch gives each responsibility an explicit object or protocol.** This chapter develops those pieces through one recurring task: a multilayer perceptron that classifies Fashion-MNIST images.

By the end of this chapter, you should be able to

1. Create, inspect, reshape, index, reduce, and broadcast PyTorch tensors.
2. Predict the shape and dtype produced by common tensor operations.
3. Explain what autograd records and where gradients are stored.
4. Define a model by subclassing `nn.Module` and inspect its registered parameters.
5. Separate sample access (`Dataset`) from batching and iteration (`DataLoader`).
6. Implement correct training and validation loops.
7. Aggregate metrics correctly when batches have different sizes.
8. Save, restore, and use a model through its `state_dict`.

0.1 The system we will build

The final program will implement the familiar five-part learning frame:

Learning component	PyTorch representation in this chapter
Data	`FashionMNIST`, transforms, and `DataLoader`
Hypothesis	`FashionMLP(nn.Module)`
Loss	`nn.CrossEntropyLoss()`
Optimization	autograd plus `torch.optim.SGD`
Evaluation	validation loss and classification accuracy

The point is not the sophistication of the network. Students already know multilayer perceptrons. The point is to make the *software contract* of each component visible.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2
```

The examples assume a recent PyTorch and TorchVision release. The official PyTorch Learn the Basics tutorial is an excellent companion reference, and the official API documentation should be the first place you check when the exact behavior of an operation matters.

1 From arrays to tensors

1.1 A tensor carries a contract

A tensor is an n-dimensional array together with metadata and runtime behavior. Four properties should become automatic debugging questions:

- **shape:** how many axes exist, and what does each axis mean?
- **dtype:** what kind of values are represented?
- **device:** where is the storage located?
- **gradient status:** should autograd track operations involving this tensor?

```
image = torch.zeros(1, 28, 28)

print(image.shape)          # torch.Size([1, 28, 28])
print(image.dtype)          # torch.float32
print(image.device)         # device(type='cpu')
print(image.requires_grad)  # False
```

For this image, the axes mean `(channel, height, width)`. The shape is not merely a storage detail: it is part of the interface expected by the model.

1.1.1 Scalars, vectors, matrices, and higher-rank tensors

```
scalar = torch.tensor(3.0)                    # shape: ()
vector = torch.tensor([1.0, 2.0, 3.0])        # shape: (3,)
matrix = torch.tensor([[1.0, 2.0],
                       [3.0, 4.0]])           # shape: (2, 2)
images = torch.zeros(64, 1, 28, 28)           # shape: (64, 1, 28, 28)
```

The number of axes is the rank, available through `tensor.ndim`.

```
scalar.ndim, vector.ndim, matrix.ndim, images.ndim
# (0, 1, 2, 4)
```

`numel()` counts scalar values, independently of shape.

```
images.numel()
# 50176 = 64 * 1 * 28 * 28
```

1.2 Creating tensors

1.2.1 From Python data

```
labels = torch.tensor([6, 0, 9, 2])
scores = torch.tensor([0.25, 0.70, 0.05])
table = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
])
```

PyTorch infers a dtype from the values:

```
labels.dtype  # torch.int64
scores.dtype  # torch.float32
```

Specify it when the intended representation is not obvious.

```
binary_targets = torch.tensor([0, 1, 1, 0], dtype=torch.float32)
class_targets = torch.tensor([0, 1, 1, 0], dtype=torch.long)
```

The distinction matters: a binary loss may expect floating targets, whereas `CrossEntropyLoss` expects integer class indices of dtype `torch.long`.

1.2.2 Filled tensors

```
zeros = torch.zeros(2, 3)
ones = torch.ones(2, 3)
constant = torch.full((2, 3), 7.5)
identity = torch.eye(4)
```

Use the `*_like` forms when an existing tensor already has the correct shape, dtype, and device.

```
reference = torch.empty(64, 10, device="cpu")

mask_buffer = torch.zeros_like(reference)
unit_buffer = torch.ones_like(reference)
```

This avoids repeating metadata and is particularly useful in device-independent code.

1.2.3 Ranges and evenly spaced values

```
indices = torch.arange(0, 10, 2)
# tensor([0, 2, 4, 6, 8])

grid = torch.linspace(0.0, 1.0, steps=5)
# tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

`arange` is defined by a step; `linspace` is defined by the number of points, including both endpoints.

1.2.4 Random tensors and reproducibility

```
torch.manual_seed(7)

uniform = torch.rand(2, 3)       # values in [0, 1)
normal = torch.randn(2, 3)       # standard normal
classes = torch.randint(0, 10, (8,))
permutation = torch.randperm(8)
```

Resetting the seed reproduces the same pseudorandom sequence in the same environment:

```
torch.manual_seed(7)
a = torch.randn(3)

torch.manual_seed(7)
b = torch.randn(3)

torch.equal(a, b)  # True
```

A seed is necessary for reproducibility, but it is not always sufficient for bitwise-identical accelerator results. Consult PyTorch's reproducibility notes when exact repeatability matters.

1.3 Moving between NumPy and PyTorch

```
import numpy as np

array = np.array([1.0, 2.0, 3.0], dtype=np.float32)

tensor_copy = torch.tensor(array)
tensor_shared = torch.from_numpy(array)
```

`torch.tensor(array)` copies the values. `torch.from_numpy(array)` normally shares CPU memory with the NumPy array.

```
array[0] = 99.0

tensor_copy
# tensor([1., 2., 3.])

tensor_shared
# tensor([99.,  2.,  3.])
```

The reverse direction can also share memory when the tensor is on the CPU and has a compatible dtype.

```
tensor = torch.tensor([10.0, 20.0, 30.0])
array_view = tensor.numpy()

tensor[1] = -5.0
array_view
# array([10., -5., 30.], dtype=float32)
```

When you need an independent object, make the copy explicit.

```
safe_array = tensor.detach().cpu().numpy().copy()
safe_tensor = torch.from_numpy(array.copy())
```

> ⚠ **Aliasing is observable behavior**
>
> Shared storage is efficient, but modifying one object can silently modify the other. Before crossing the NumPy/PyTorch boundary, decide whether you want a view or an independent copy.

2 Tensor fluency

2.1 Indexing and slicing

PyTorch follows NumPy's indexing conventions. Each axis receives an index or a slice.

```
x = torch.arange(24).reshape(2, 3, 4)
# shape: (2, 3, 4)
```

An integer index removes an axis:

```
x[0].shape       # torch.Size([3, 4])
x[:, 1].shape    # torch.Size([2, 4])
x[:, :, 2].shape # torch.Size([2, 3])
```

A length-one slice preserves it:

```
x[0:1].shape       # torch.Size([1, 3, 4])
x[:, 1:2].shape    # torch.Size([2, 1, 4])
x[:, :, 2:3].shape # torch.Size([2, 3, 1])
```

Other useful patterns are:

```
x[..., -1]      # final element along the last axis
x[:, ::2, :]    # every second element along axis 1
x[:, :, 1:]     # omit the first item along the last axis
x[1, :, [0, 3]] # advanced indexing with selected columns
```

2.1.1 Why preserving an axis matters

Suppose a model expects a batch axis.

```
batch = torch.randn(64, 1, 28, 28)

one_image = batch[0]       # (1, 28, 28): batch axis removed
one_batch = batch[0:1]     # (1, 1, 28, 28): batch axis preserved
```

Both contain one image, but they do not have the same interface. Alternatively, restore the missing axis:

```
one_batch = one_image.unsqueeze(0)
```

2.2 Boolean masks

A comparison creates a Boolean tensor with the same shape as its input.

```
confidence = torch.tensor([0.10, 0.82, 0.43, 0.91, 0.67])
mask = confidence ⩾ 0.60

mask
# tensor([False, True, False, True, True])
```

Use the mask to select values:

```
selected = confidence[mask]
# tensor([0.8200, 0.9100, 0.6700])
```

Or use it to update matching locations:

```
clipped = confidence.clone()
clipped[clipped < 0.50] = 0.0
# tensor([0.0000, 0.8200, 0.0000, 0.9100, 0.6700])
```

`torch.where` chooses between two values without changing the shape:

```
status = torch.where(
    confidence ⩾ 0.60,
    torch.tensor(1),
    torch.tensor(0),
)
# tensor([0, 1, 0, 1, 1])
```

Notice that Boolean selection usually produces a one-dimensional result because the number of selected elements is data-dependent.

2.3 Reshaping without changing the values

```
pixels = torch.arange(12)

pixels.reshape(3, 4)
# tensor([[ 0,  1,  2,  3],
#         [ 4,  5,  6,  7],
#         [ 8,  9, 10, 11]])
```

The number of elements must be preserved. One dimension may be inferred with `-1`.

```
pixels.reshape(2, -1).shape  # torch.Size([2, 6])
pixels.reshape(-1, 3).shape  # torch.Size([4, 3])
```

For a batch of images, keep the first axis and flatten the rest:

```
images = torch.randn(64, 1, 28, 28)
flat = images.reshape(images.size(0), -1)

flat.shape  # torch.Size([64, 784])
```

`flatten` states the same intent more directly:

```
flat = images.flatten(start_dim=1)
```

2.3.1 `reshape`, `view`, and contiguity

`view` requires a compatible memory layout. Operations such as `transpose` can produce a non-contiguous tensor.

```
x = torch.arange(12).reshape(3, 4)
y = x.transpose(0, 1)

y.is_contiguous()  # False
```

This can fail:

```
# y.view(12)  # RuntimeError for an incompatible stride layout
```

`reshape` returns a view when possible and otherwise makes a copy:

```
z = y.reshape(12)
```

For most application code, prefer `reshape` unless sharing and stride behavior are specifically important.

2.4 Adding, removing, and reordering axes

```
x = torch.randn(28, 28)

x.unsqueeze(0).shape  # (1, 28, 28)
x.unsqueeze(2).shape  # (28, 28, 1)
```

`squeeze` removes axes of size one:

```
batch = torch.randn(1, 1, 28, 28)

batch.squeeze(0).shape  # (1, 28, 28)
batch.squeeze().shape   # (28, 28)
```

Prefer specifying the dimension. An unrestricted `squeeze()` may accidentally remove a batch axis when the batch happens to contain one example.

Reorder axes with `permute`:

```
nchw = torch.randn(64, 3, 28, 28)
nhwc = nchw.permute(0, 2, 3, 1)

nhwc.shape  # torch.Size([64, 28, 28, 3])
```

Swap exactly two axes with `transpose`:

```
matrix = torch.randn(3, 5)
matrix.transpose(0, 1).shape  # torch.Size([5, 3])
matrix.T.shape                # torch.Size([5, 3])
```

2.5 Reductions and dimensions

Consider a score matrix whose rows are examples and columns are classes:

```
scores = torch.tensor([
    [0.1, 0.7, 0.2],
    [0.8, 0.1, 0.1],
])  # shape: (2 examples, 3 classes)
```

Reducing `dim=0` removes the row axis and produces one result per column:

```
scores.mean(dim=0)
# tensor([0.4500, 0.4000, 0.1500])  shape: (3,)
```

Reducing `dim=1` removes the column axis and produces one result per row:

```
scores.sum(dim=1)
# tensor([1., 1.])  shape: (2,)
```

Reducing both axes produces a scalar:

```
scores.mean(dim=(0, 1))
# tensor(0.3333)  shape: ()
```

Preserve reduced axes with `keepdim=True` when the result must broadcast back against the input:

```
row_mean = scores.mean(dim=1, keepdim=True)
row_mean.shape  # torch.Size([2, 1])

centered = scores - row_mean
centered.shape  # torch.Size([2, 3])
```

2.5.1 Values and indices

`max` and `min` with a dimension return both values and positions.

```
values, indices = scores.max(dim=1)

values   # tensor([0.7000, 0.8000])
indices  # tensor([1, 0])
```

If only the index matters, use `argmax`:

```
predicted_classes = scores.argmax(dim=1)
```

2.6 Broadcasting

Broadcasting lets tensors with compatible shapes participate in elementwise operations without explicitly copying data. Compare shapes from the rightmost dimension toward the left. Two dimensions are compatible when they are equal or one of them is 1; missing leading dimensions behave like 1.

2.6.1 Scalar with a vector

```
x = torch.tensor([1.0, 2.0, 3.0, 4.0])  # (4,)
y = x + 10.0                              # (4,) + () -> (4,)
```

Conceptually, the scalar behaves like `[10, 10, 10, 10]` for this operation.

2.6.2 Row vector with a matrix

```
x = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
])                                        # (2, 3)
bias = torch.tensor([10.0, 20.0, 30.0]) # (3,)

x + bias
# tensor([[11., 22., 33.],
#         [14., 25., 36.]])
```

Shape comparison:

```
  x:     (2, 3)
  bias:     (3)
  result: (2, 3)
```

2.6.3 Column vector with a matrix

```
offset = torch.tensor([[100.0],
                       [200.0]])  # (2, 1)

x + offset
# tensor([[101., 102., 103.],
#         [204., 205., 206.]])
```

Shape comparison:

```
  x:      (2, 3)
  offset: (2, 1)
  result: (2, 3)
```

2.6.4 Per-channel image normalization

For a batch `(N, C, H, W)`, a channel statistic often begins as `(C,)`. It must be reshaped so the channel aligns with axis 1 rather than the final axis.

```
images = torch.randn(64, 3, 28, 28)
mean = torch.tensor([0.48, 0.45, 0.40])
std = torch.tensor([0.23, 0.22, 0.24])

mean = mean.reshape(1, 3, 1, 1)
std = std.reshape(1, 3, 1, 1)

normalized = (images - mean) / std
normalized.shape  # torch.Size([64, 3, 28, 28])
```

The size-one batch and spatial axes expand conceptually:

```
images: (64, 3, 28, 28)
mean:   ( 1, 3,  1,  1)
std:    ( 1, 3,  1,  1)
result: (64, 3, 28, 28)
```

Read the official broadcasting semantics when debugging a difficult shape combination.

> **A reliable broadcasting habit**
>
> Write the shapes one above another, right-aligned. Do not reason from the variable names. Broadcasting operates on sizes and axis positions, not on what you intended the axes to mean.

2.7 Practice: predict before running

For each expression, predict the output shape or explain why it fails.

```
a = torch.zeros(8, 1, 5)
b = torch.zeros(1, 4, 5)
c = torch.zeros(8, 4)
```

1. `a + b`
2. `a.mean(dim=2)`
3. `a.mean(dim=2, keepdim=True)`
4. `a + c`
5. `a.squeeze(1) + c`

Worked solution

The results are `(8,4,5)`, `(8,1)`, `(8,1,1)`, an incompatibility error, and another incompatibility error respectively.

3 Autograd

3.1 What the forward pass records

Autograd records operations involving tensors that require gradients. Consider a one-example regression model:

$$\hat{y} = wx + b, \qquad L = (\hat{y} - y)^2.$$

```
x = torch.tensor(2.0)
target = torch.tensor(10.0)

w = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)

prediction = w * x + b
loss = (prediction - target) ** 2

prediction  # tensor(7., grad_fn=<AddBackward...>)
loss        # tensor(9., grad_fn=<PowBackward...>)
```

The forward pass computes values and constructs a dynamic computation graph. The graph is dynamic because ordinary Python control flow can determine which operations run.

```
def piecewise(x):
    if x.item() ⩾ 0:
        return x ** 2
    return 3 * x
```

Each call records only the operations actually executed.

3.2 Backward populates leaf gradients

```
loss.backward()

w.grad  # tensor(-12.)
b.grad  # tensor(-6.)
```

The derivatives are

$$\frac{\partial L}{\partial w} = 2(\hat{y} - y)x = 2(7 - 10)(2) = -12,$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y) = -6.$$

`w` and `b` are leaf tensors: the program created them directly, and they require gradients. Intermediate tensors normally participate in gradient computation without retaining their own `.grad` buffers.

```
w.is_leaf           # True
prediction.is_leaf  # False
prediction.grad     # normally None
```

Use `.retain_grad()` only when debugging requires an intermediate gradient.

```
w = torch.tensor(3.0, requires_grad=True)
prediction = w * 2
prediction.retain_grad()
loss = (prediction - 10) ** 2
loss.backward()

prediction.grad  # tensor(-8.)
```

3.3 A batch loss becomes one scalar

```
prediction = torch.tensor(
    [4.0, 3.0, 10.0],
    requires_grad=True,
)
target = torch.tensor([3.0, 5.0, 7.0])

per_example = (prediction - target) ** 2
loss = per_example.mean()

per_example  # tensor([1., 4., 9.])
loss         # tensor(4.6667)
```

Calling `backward()` without an explicit argument is natural for a scalar objective:

```
loss.backward()
prediction.grad
# tensor([ 0.6667, -1.3333,  2.0000])
```

The factor $1/3$ comes from the mean. If the loss used a sum, the gradient would be three times larger.

3.4 Gradients accumulate

```
w = torch.tensor(2.0, requires_grad=True)

(w ** 2).backward()
w.grad  # tensor(4.)

(w ** 2).backward()
w.grad  # tensor(8.)
```

The second call adds into the existing gradient buffer. Clear it before the next independent training batch:

```
w.grad.zero_()
w.grad  # tensor(0.)
```

Optimizers provide `zero_grad()` for all registered parameters:

```
optimizer.zero_grad()
```

Using `set_to_none=True` can avoid writing zeros and lets `None` signal that no gradient was produced:

```
optimizer.zero_grad(set_to_none=True)
```

3.5 Disabling gradient recording

Parameter updates and evaluation do not need a computation graph.

```
with torch.no_grad():
    w -= 0.1 * w.grad
```

For evaluation-only code, `torch.inference_mode()` is a stronger optimization:

```
with torch.inference_mode():
    prediction = model(images)
```

`model.eval()` and `torch.inference_mode()` solve different problems:

Mechanism	What it changes
`model.eval()`	behavior of stateful/stochastic modules such as BatchNorm and Dropout
`torch.inference_mode()`	autograd recording and related tensor bookkeeping

Use both during validation and deployment inference.

For a deeper treatment, see PyTorch's automatic differentiation tutorial and autograd mechanics note.

4 Models as `nn.Module`

4.1 The module contract

`nn.Module` is PyTorch's base class for models and layers. A custom model normally implements two methods:

- `__init__`: create and assign reusable submodules or parameters;
- `forward`: define how inputs flow through those components.

The base class supplies the surrounding machinery: recursive parameter registration, device movement, training/evaluation modes, hooks, state dictionaries, and the callable `model(x)` interface.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = ...

    def forward(self, x):
        output = ...
        return output
```

Always call `super().__init__()` before assigning submodules.

4.2 A Fashion-MNIST MLP

```python
class FashionMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.network = nn.Sequential(
            nn.Linear(28 * 28, 128),
            nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, images):
        x = self.flatten(images)
        logits = self.network(x)
        return logits
```

The shape path is

```
(B, 1, 28, 28) -> (B, 784) -> (B, 128) -> (B, 128) -> (B, 10)
       image         flat       linear       ReLU        logits
```

The output contains raw logits. Do not place a softmax after the final linear layer when training with `nn.CrossEntropyLoss`, because that loss applies the appropriate log-softmax internally.

4.2.1 Calling the model

```python
model = FashionMLP()
images = torch.randn(64, 1, 28, 28)
logits = model(images)

logits.shape  # torch.Size([64, 10])
```

Use `model(images)`, not `model.forward(images)`. Calling the module lets PyTorch run its surrounding machinery, including registered hooks.

4.2.2 `Sequential` versus an explicit `forward`

This model can also be written entirely as a pipeline:

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
)
```

`nn.Sequential` is appropriate when every output becomes the next module's only input. Use an explicit subclass when the computation branches, reuses layers, combines multiple inputs, returns multiple outputs, or contains other custom logic.

4.3 Registered parameters

Assigning a module to `self.*` registers it as a child. PyTorch can then recursively discover its parameters.

```
model = FashionMLP()

for name, parameter in model.named_parameters():
    print(name, tuple(parameter.shape))
```

```
network.0.weight (128, 784)
network.0.bias   (128,)
network.2.weight (10, 128)
network.2.bias   (10,)
```

The parameter count is

$$(128 \times 784) + 128 + (10 \times 128) + 10 = 101{,}770.$$

Verify it programmatically:

```
trainable = sum(
    parameter.numel()
    for parameter in model.parameters()
    if parameter.requires_grad
)

trainable  # 101770
```

`Flatten` and `ReLU` have no trainable parameters. They are still modules because a module may implement behavior without owning learned tensors.

4.4 Device movement

```
device = (
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)

model = FashionMLP().to(device)
images = images.to(device)
logits = model(images)
```

`model.to(device)` moves registered parameters and buffers recursively. It works because the child layers were assigned as submodules.

Tensor `.to(...)` returns a tensor; reassign it:

```
images = images.to(device)
```

This expression does not change the original variable:

```
images.to(device)  # returned tensor is discarded
```

> ⚠ **Devices must agree**
>
> An operation cannot normally combine a CPU tensor and a CUDA tensor. Move the model and each input batch to the same device. Optimizer state will follow the model when the optimizer is created after device placement.

5 Data pipeline

5.1 `Dataset` and `DataLoader` solve different problems

A `Dataset` represents indexed samples. A map-style dataset supports:

```
len(dataset)
image, label = dataset[index]
```

A `DataLoader` wraps a dataset with an iteration policy:

```
loader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
)

for images, labels in loader:
    ...
```

This separation keeps file access, sample transformation, shuffling, batching, and training logic from becoming one monolithic loop. The official Datasets & DataLoaders tutorial develops both abstractions further.

5.2 Loading Fashion-MNIST

```
transform = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
])

train_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=transform,
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=transform,
)
```

The constructor arguments express distinct decisions:

Argument	Meaning
`root="data"`	local directory for dataset files
`train=True/False`	choose the 60,000-example training split or 10,000-example test split
`download=True`	download files if they are absent
`transform=...`	convert each loaded image before returning it

`ToDtype(torch.float32, scale=True)` converts byte-valued image data to floating point and scales it to the usual `[0,1]` range.

5.3 Inspect a sample first

```
class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot",
]

image, label = train_data[0]

print(image.shape)              # torch.Size([1, 28, 28])
print(image.dtype)              # torch.float32
print(image.min(), image.max())
print(label, class_names[label])
```

Before constructing a loader, verify:

1. the axes have the expected meaning;
2. the dtype matches the model and loss;
3. the value range matches the intended preprocessing;
4. the label encoding matches the loss;
5. the class-name mapping is correct.

Many training failures are data-contract failures that become harder to see after samples are batched.

5.4 Batching and shuffling

```
train_loader = DataLoader(
    train_data,
    batch_size=64,
    shuffle=True,
)
```

```
test_loader = DataLoader(
    test_data,
    batch_size=64,
    shuffle=False,
)
```

For a map-style dataset, shuffling changes the index order used during an epoch. It does not rearrange the stored dataset. The loader then groups indices into batches.

```
dataset indices:  0  1  2  3  4  5  6  7
shuffled order:   5  1  7  0  3  6  2  4
batches of four: [5  1  7  0] [3  6  2  4]
```

The final batch can be smaller. Fashion-MNIST has 60,000 training examples:

```
60000 % 64  # 32
```

Therefore the final batch contains 32 examples when `drop_last=False`, which is the default.

```
for images, labels in train_loader:
    print(images.shape)  # usually (64, 1, 28, 28)
    print(labels.shape)  # usually (64,)
```

Write code in terms of the current batch size `images.size(0)`, not the configured maximum.

5.4.1 Loader performance options

Begin with the simplest loader. Add performance options only after measuring a bottleneck.

```
train_loader = DataLoader(
    train_data,
    batch_size=64,
    shuffle=True,
    num_workers=4,
    pin_memory=torch.cuda.is_available(),
)
```

- `num_workers>0` loads samples in subprocesses;
- `pin_memory=True` can accelerate CPU-to-CUDA transfer;

- `persistent_workers=True` keeps workers alive across epochs;
- a custom `collate_fn` controls how samples are combined.

Platform, dataset, storage, and batch cost determine whether these options help. See the `DataLoader` API for the complete contract.

5.5 A minimal custom dataset

Understanding the protocol makes prebuilt datasets less mysterious.

```
from pathlib import Path
from PIL import Image


class ImageTableDataset(torch.utils.data.Dataset):
    def __init__(self, rows, image_dir, transform=None):
        self.rows = rows.reset_index(drop=True)
        self.image_dir = Path(image_dir)
        self.transform = transform

    def __len__(self):
        return len(self.rows)

    def __getitem__(self, index):
        row = self.rows.iloc[index]
        path = self.image_dir / row["filename"]
        image = Image.open(path).convert("L")

        if self.transform is not None:
            image = self.transform(image)

        label = int(row["label"])
        return image, label
```

The dataset does not create batches. It answers one indexed request. The loader handles sampling, batching, and collation.

6 Training and validation

6.1 Configure the objective and optimizer

```
model = FashionMLP().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=1e-3,
)
```

`CrossEntropyLoss` expects:

- logits with shape `(B, C)` and floating dtype;
- class indices with shape `(B,)` and integer dtype;
- labels in the range $0 \leqslant \texttt{label} < C$.

```
images, labels = next(iter(train_loader))
images = images.to(device)
labels = labels.to(device)

logits = model(images)
loss = loss_fn(logits, labels)

logits.shape  # (B, 10)
labels.shape  # (B,)
loss.shape    # ()
```

Common mistakes include applying softmax before the loss, one-hot encoding labels unnecessarily, or passing floating labels.

6.2 One optimization step

```
images = images.to(device)
labels = labels.to(device)

optimizer.zero_grad()

logits = model(images)
loss = loss_fn(logits, labels)

loss.backward()
optimizer.step()
```

The order describes state transitions:

1. `zero_grad()` clears gradients left by previous batches;
2. the forward pass records a new graph;

3. the loss creates a scalar objective;
4. `backward()` writes parameter gradients;
5. `step()` reads those gradients and changes parameters.

Forgetting `zero_grad()` unintentionally accumulates gradients. Calling `step()` before `backward()` uses missing or stale gradients.

6.3 Training for one epoch

```
def train_one_epoch(model, loader, loss_fn, optimizer, device):
    model.train()
    total_loss = 0.0
    total_correct = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        logits = model(images)
        loss = loss_fn(logits, labels)
        loss.backward()
        optimizer.step()

        batch_size = images.size(0)
        total_loss += loss.item() * batch_size
        total_correct += (
            logits.argmax(dim=1) == labels
        ).sum().item()

    n = len(loader.dataset)
    return {
        "loss": total_loss / n,
        "accuracy": total_correct / n,
    }
```

`loss.item()` converts a scalar tensor into a Python number. Do not accumulate the tensor itself across the epoch: retaining graph-connected tensors can keep unnecessary computation graphs alive.

6.4 Validation

```python
def validate(model, loader, loss_fn, device):
    model.eval()
    total_loss = 0.0
    total_correct = 0

    with torch.inference_mode():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            logits = model(images)
            loss = loss_fn(logits, labels)

            batch_size = images.size(0)
            total_loss += loss.item() * batch_size
            total_correct += (
                logits.argmax(dim=1) == labels
            ).sum().item()

    n = len(loader.dataset)
    return {
        "loss": total_loss / n,
        "accuracy": total_correct / n,
    }
```

Validation differs from training in three essential ways:

Concern	Training	Validation
module mode	`model.train()`	`model.eval()`
gradient recording	enabled	`torch.inference_mode()`
parameter update	`optimizer.step()`	none

Validation measures a frozen training state. If validation code updates parameters, it is no longer validation.

6.5 Correct metric aggregation

`CrossEntropyLoss()` returns a mean over the current batch by default. Averaging those batch means is wrong when batch sizes differ.

Suppose one batch has 64 examples with mean loss 0.50 and the final batch has 16 examples with mean loss 1.00. The mean of batch means is

$$\frac{0.50 + 1.00}{2} = 0.75,$$

but the example-weighted mean is

$$\frac{64(0.50) + 16(1.00)}{64 + 16} = 0.60.$$

The implementation therefore accumulates loss sums:

```
total_loss += loss.item() * images.size(0)
mean_loss = total_loss / len(loader.dataset)
```

Accuracy already begins as a count:

```
total_correct += (
    logits.argmax(dim=1) == labels
).sum().item()

accuracy = total_correct / len(loader.dataset)
```

6.5.1 Confusion matrices and per-class behavior

Overall accuracy can hide systematic failures. Accumulate predictions and labels during validation:

```
all_predictions = []
all_targets = []

model.eval()
with torch.inference_mode():
    for images, labels in test_loader:
        logits = model(images.to(device))
        predictions = logits.argmax(dim=1).cpu()

        all_predictions.append(predictions)
        all_targets.append(labels)

all_predictions = torch.cat(all_predictions)
all_targets = torch.cat(all_targets)
```

A confusion matrix counts `(true class, predicted class)` pairs:

```
confusion = torch.zeros(10, 10, dtype=torch.long)

for target, prediction in zip(all_targets, all_predictions):
    confusion[target, prediction] += 1
```

Inspecting confusion between `Shirt`, `T-shirt/top`, `Pullover`, and `Coat` is often more informative than a single accuracy value.

6.6 Complete epoch driver

```
epochs = 10
history = []

for epoch in range(1, epochs + 1):
    train_metrics = train_one_epoch(
        model,
        train_loader,
        loss_fn,
        optimizer,
        device,
    )
    val_metrics = validate(
        model,
        test_loader,
        loss_fn,
        device,
    )

    record = {
        "epoch": epoch,
        "train_loss": train_metrics["loss"],
        "train_accuracy": train_metrics["accuracy"],
        "val_loss": val_metrics["loss"],
        "val_accuracy": val_metrics["accuracy"],
    }
    history.append(record)

    print(
        f"epoch {epoch:02d} | "
        f"train loss {record['train_loss']:.4f} | "
        f"val loss {record['val_loss']:.4f} | "
        f"val accuracy {record['val_accuracy']:.1%}"
    )
```

Keeping structured history makes plots, comparisons, and checkpoint decisions possible without parsing console text.

> **About the test split**
>
> For a classroom demonstration, the official Fashion-MNIST test split is often used as a validation set. In a serious experiment, create a validation split from the training data, use it for model selection, and reserve the test set for one final unbiased report.

6.7 Practice: repair a broken loop

Find at least five problems.

```
def broken_validate(model, loader, optimizer):
    model.train()
    correct = 0

    for images, labels in loader:
        logits = model(images)
        loss = nn.CrossEntropyLoss()(logits.softmax(dim=1),
↪   labels.float())
        loss.backward()
        optimizer.step()
        correct += (logits.argmax(dim=0) == labels).sum()

    return correct / len(loader)
```

> **Worked solution**
>
> Use evaluation mode and inference mode; remove the optimizer and backward pass; move both tensors to the device; pass raw logits and integer labels to the loss; use `argmax(dim=1)`; accumulate integer correct counts; and divide by the dataset size.

7 Saving, restoring, and inference

7.1 Save the best validation state

The final epoch is not necessarily the best epoch. Save a checkpoint when the chosen validation metric improves.

```
best_val_loss = float("inf")

for epoch in range(1, epochs + 1):
    train_metrics = train_one_epoch(
```

```
        model, train_loader, loss_fn, optimizer, device
    )
    val_metrics = validate(
        model, test_loader, loss_fn, device
    )

    if val_metrics["loss"] < best_val_loss:
        best_val_loss = val_metrics["loss"]
        torch.save(
            model.state_dict(),
            "fashion_mlp.pth",
        )
```

Choose the selection metric before training. Validation loss is a natural default because it matches the training objective, but a task may prioritize accuracy, recall, calibration, or another operational measure.

7.2 What a `state_dict` contains

```
state = model.state_dict()

for name, value in state.items():
    print(name, tuple(value.shape))
```

```
network.0.weight (128, 784)
network.0.bias   (128,)
network.2.weight (10, 128)
network.2.bias   (10,)
```

A state dictionary maps registered names to tensor values. It does not by itself define the Python architecture.

```
torch.save(state, "fashion_mlp.pth")
```

Saving a `state_dict` is PyTorch's recommended flexible pattern for model weights. See the official Saving and Loading Models tutorial.

7.3 Restore on any chosen device

```
restored = FashionMLP().to(device)

state = torch.load(
    "fashion_mlp.pth",
    map_location=device,
    weights_only=True,
)

restored.load_state_dict(state)
restored.eval()
```

Three pieces are required:

1. the class definition recreates the architecture;
2. `torch.load` reads tensor state and maps storage to a device;
3. `load_state_dict` matches saved names and shapes to the model.

The return value reports missing or unexpected keys. Do not ignore mismatches unless you intentionally perform partial loading.

7.3.1 Saving enough information to resume training

Weights alone are sufficient for inference. Resuming an interrupted training run usually requires more state:

```
checkpoint = {
    "epoch": epoch,
    "model_state": model.state_dict(),
    "optimizer_state": optimizer.state_dict(),
    "best_val_loss": best_val_loss,
    "history": history,
}

torch.save(checkpoint, "training_checkpoint.pth")
```

Restore both objects:

```
checkpoint = torch.load(
    "training_checkpoint.pth",
    map_location=device,
    weights_only=False,
)

model = FashionMLP().to(device)
model.load_state_dict(checkpoint["model_state"])
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
optimizer.load_state_dict(checkpoint["optimizer_state"])

start_epoch = checkpoint["epoch"] + 1
best_val_loss = checkpoint["best_val_loss"]
history = checkpoint["history"]
```

Only load general pickled checkpoints from trusted sources. `weights_only=True` is preferred when the file contains only tensor weights.

7.4 Predict one image

```
model = FashionMLP().to(device)
state = torch.load(
    "fashion_mlp.pth",
    map_location=device,
    weights_only=True,
)
model.load_state_dict(state)
model.eval()

image, target = test_data[0]
image_batch = image.unsqueeze(0).to(device)

with torch.inference_mode():
    logits = model(image_batch)
    predicted = logits.argmax(dim=1).item()

print("predicted:", class_names[predicted])
print("target:", class_names[target])
```

The dataset returns one image with shape `(1,28,28)`. `unsqueeze(0)` creates the batch shape `(1,1,28,28)` required by the model.

If probabilities are needed for presentation or calibration analysis, compute them after obtaining logits:

```
probabilities = logits.softmax(dim=1)
confidence, predicted = probabilities.max(dim=1)
```

Do not confuse confidence with correctness. A model can assign high probability to a wrong prediction.

8 A complete minimal program

The following listing combines the chapter's components without hiding the important framework calls.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2


class FashionMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.network = nn.Sequential(
            nn.Linear(28 * 28, 128),
            nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, images):
        return self.network(self.flatten(images))


def train_one_epoch(model, loader, loss_fn, optimizer, device):
    model.train()
    total_loss = 0.0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        logits = model(images)
        loss = loss_fn(logits, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * images.size(0)

    return total_loss / len(loader.dataset)


def validate(model, loader, loss_fn, device):
    model.eval()
```

```
    total_loss = 0.0
    total_correct = 0

    with torch.inference_mode():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)
            logits = model(images)

            total_loss += (
                loss_fn(logits, labels).item()
                * images.size(0)
            )
            total_correct += (
                logits.argmax(dim=1) == labels
            ).sum().item()

    n = len(loader.dataset)
    return total_loss / n, total_correct / n


device = "cuda" if torch.cuda.is_available() else "cpu"

transform = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
])

train_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=transform,
)
test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=transform,
)

train_loader = DataLoader(
    train_data,
    batch_size=64,
    shuffle=True,
)
test_loader = DataLoader(
```

```
    test_data,
    batch_size=64,
    shuffle=False,
)

model = FashionMLP().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)

best_val_loss = float("inf")

for epoch in range(1, 11):
    train_loss = train_one_epoch(
        model, train_loader, loss_fn, optimizer, device
    )
    val_loss, val_accuracy = validate(
        model, test_loader, loss_fn, device
    )

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), "fashion_mlp.pth")

    print(
        f"epoch {epoch:02d} | "
        f"train loss {train_loss:.4f} | "
        f"val loss {val_loss:.4f} | "
        f"val accuracy {val_accuracy:.1%}"
    )
```

9 Debugging checklist

When a PyTorch program fails, inspect contracts in this order.

9.1 Data

```
images, labels = next(iter(train_loader))

print(images.shape, images.dtype, images.device)
print(labels.shape, labels.dtype, labels.device)
print(images.min(), images.max())
print(labels.min(), labels.max())
```

Questions:

- Is there a batch axis?
- Are class labels integers in `[0,C)`?
- Did preprocessing create the intended value range?
- Are images and labels paired correctly?

9.2 Model

```
print(model)
print(sum(p.numel() for p in model.parameters()))

logits = model(images.to(device))
print(logits.shape, logits.dtype, logits.device)
```

Questions:

- Does the output preserve the batch axis?
- Does the final axis equal the number of classes?
- Are raw logits returned?

9.3 Gradients

```
loss = loss_fn(logits, labels.to(device))
optimizer.zero_grad()
loss.backward()

for name, parameter in model.named_parameters():
    if parameter.grad is None:
        print(name, "NO GRADIENT")
    else:
        print(name, parameter.grad.norm().item())
```

Questions:

- Did every intended parameter receive a gradient?
- Are gradient norms finite?
- Did a tensor become detached unexpectedly?
- Were gradients cleared at the intended time?

9.4 Evaluation

Questions:

- Was `model.eval()` called?
- Was autograd disabled?
- Is the validation set excluded from optimization?
- Are metric totals divided by examples rather than batches?
- Was the selected checkpoint actually restored before final evaluation?

10 Further reading

- PyTorch · Learn the Basics: the official end-to-end beginner sequence.
- PyTorch · Tensor tutorial: tensor creation, attributes, indexing, operations, NumPy interoperability.
- PyTorch · Broadcasting semantics: precise compatibility rules.
- PyTorch · Autograd tutorial: dynamic graphs and gradient computation.
- PyTorch · `nn.Module`: the complete module API.
- PyTorch · Datasets & DataLoaders: sample access and batching.
- PyTorch · Optimization tutorial: training and evaluation loops.
- PyTorch · Saving and loading models: inference weights and resumable checkpoints.
- Stanford CS224N · PyTorch Tutorial: a compact complementary introduction with many small examples.

11 Chapter summary

The central ideas are contracts and state:

1. A tensor's shape, dtype, device, and gradient status determine which operations are valid.
2. Indexing, reshaping, reductions, and broadcasting change shapes according to explicit rules.
3. Autograd records the executed forward computation and accumulates gradients into leaves.
4. `nn.Module` registers a model's reusable components and exposes them recursively.
5. `Dataset` returns individual samples; `DataLoader` chooses order and forms batches.
6. Training enables gradients and updates parameters; validation changes module mode, disables gradients, and freezes parameters.
7. Metrics must be aggregated over examples when batch sizes differ.
8. A `state_dict` stores registered values; the Python class supplies the architecture.

Once these contracts become habitual, PyTorch stops feeling like a collection of special commands. It becomes a coherent way to represent a learning system whose data, computation, state changes, and evaluation decisions remain visible.